
系统开发工具基础

第 2 周实验报告

课程名称： 系统开发工具基础

授课教师： 周小伟

学生姓名： 洪朦朦

学 号： 24170001032

报告日期： 2026 年 9 月 6 日

GitHub: https://github.com/88594959h/Assignment_for_System_Development_Tools

1 实验概述

1.1 实验环境

- 操作系统: Windows 11 + Git Bash (MINGW64)
- Shell: Bash (Git for Windows)
- 版本控制: Git 2.x
- L^AT_EX 发行版: TeX Live 2024+
- 编辑器: VS Code

1.2 GitHub 仓库信息

- 仓库地址: https://github.com/88594959h/Assignment_for_System_Development_Tools

Commits on Sep 4, 2026	
q08-2026/9/4 88594959h committed 2 days ago	0bbc78b
q07-2026/9/4 88594959h committed 2 days ago	232f1e3
Commits on Sep 2, 2026	
q06 88594959h committed 4 days ago	1986227
q05 88594959h committed 4 days ago	6b5db6a
experiment_report 88594959h committed 4 days ago	548780c

图 1: Git Commit 历史记录截图 (git log --oneline)

2 课上实验 (第 2 周)

2.1 实验一：控制一个可清理的后台任务

2.1.1 题目要求

1. 将给定脚本保存为 `q05/worker.sh`，赋予执行权限，在前台启动并将 `stdout`、`stderr` 分别写入 `stdout.log`、`stderr.log`。
2. 使用 `Ctrl-Z` 挂起任务，再用 `bg` 让其在后台继续运行；使用 `jobs` 确认状态。
3. 使用 `jobs -p` 或等价方式动态取得 `PID`（不得手工抄写），发送 `SIGTERM` 并等待任务结束。
4. 确认 `cleanup.log` 中出现 `CLEAN_EXIT`，且任务已正常退出；禁止使用 `kill -9`。

2.1.2 核心指令与实现

```
#1. 生成脚本
$ cat > worker.sh << 'EOF'
> #!/usr/bin/env bash
trap 'echo CLEAN_EXIT >> cleanup.log; exit 0' TERM INT
n=0
while true; do
    echo "$n"
    n=$((n+1))
    sleep 1
done
EOF

#2. 赋予权限
$ chmod +x worker.sh

#3. 前台启动并重定向日志
$ ./worker.sh >stdout.log 2>stderr.log

#4. 在终端按 Ctrl-Z 挂起后，后台继续，确认状态
$ bg %1
$ jobs

#5. 动态获取 PID 并发送 SIGTERM
```

```
$ PID=$(jobs -p)
$ kill -TERM "$PID"
$ wait "$PID"

#6. 验证清理结果
$ grep "CLEAN_EXIT" cleanup.log
$ jobs
$ ps -p "$PID"
```

Listing 1: 实验一关键 Shell 指令

2.1.3 实验分析

实现思路

准备阶段创建脚本并赋权，以前台模式启动且分离重定向 stdout 与 stderr；通过 Ctrl-Z 挂起进程后使用 bg 转入后台，并用 jobs 验证状态；利用 jobs -p 动态获取 PID 发送 SIGTERM 触发 trap 清理机制，最后用 wait 等待退出并验证 cleanup.log 中的标记。

实验重点

- **I/O 重定向语法：**掌握 `>stdout.log 2>stderr.log` 分别重定向 stdout (fd=1) 和 stderr (fd=2) 的写法。
- **作业控制三件套：**理解 Ctrl-Z（挂起）、bg（后台继续）、jobs（查看状态）的协作流程。
- **动态 PID 获取：**必须使用 jobs -p 或 \$! 等 shell 内置机制获取 PID，严禁手动复制粘贴数字。
- **Trap 信号处理：**理解 trap '...' TERM INT 的作用——捕获 SIGTERM/SIGINT 后执行清理逻辑再退出，这是实现“可清理”的核心。
- **优雅退出 vs 强制杀死：**区分 SIGTERM（允许进程自行清理）与 SIGKILL/kill -9（立即终止、无法被 trap 捕获）的本质区别。

2.1.4 运行结果

```
HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools/q05 (master)
$ cat > worker.sh << 'EOF'
> #!/usr/bin/env bash
trap 'echo CLEAN_EXIT >> cleanup.log; exit 0' TERM INT
n=0
while true; do
  echo "$n"
  n=$((n+1))
  sleep 1
done
EOF
```

图 2: 实验一: 生成脚本

```
HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools/q05 (master)
$ ./worker.sh >stdout.log 2>stderr.log
[1]+  Stopped                  ./worker.sh > stdout.log 2> stderr.log
```

图 3: 实验一: 重定向日志

```
HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools/q05 (master)
$ bg %1
[1]+  ./worker.sh > stdout.log 2> stderr.log &

HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools/q05 (master)
$ jobs
[1]+  Running                  ./worker.sh > stdout.log 2> stderr.log &
```

图 4: 实验一: 挂起并确认状态

```
HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools/q05 (master)
$ grep "CLEAN_EXIT" cleanup.log
CLEAN_EXIT

HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools/q05 (master)
$ jobs

HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools/q05 (master)
$ ps -p "$PID"
  PID  PPID  PGID  WINPID  TTY      UID    STIME  COMMAND
```

图 5: 实验一: 验证清理结果

2.2 实验二: 语义重构与本地开发反馈

2.2.1 题目要求

1. 在已配置好 Python、pytest 和 ruff 的课程环境中打开 q06 目录，并启用 Python 语言服务器；按照给定内容分别创建 `math_utils.py`、`app.py` 和 `test_math_utils.py` 三个文件。
2. 使用编辑器语言服务器功能，分别演示对符号 `total_price` 的“跳转到定义”与“查找引用”操作。

3. 使用“重命名符号”功能将 `total_price` 重构为 `calculate_total`，确保函数定义及所有跨文件引用同步更新，且无遗漏或错误。
4. 在 `app.py` 中临时添加未使用的导入语句 `import os`，使用 `ruff` 检测该问题并自动/手动删除；最终运行 `ruff check` 与 `pytest`，确保两者均无报错并通过验证。

2.2.2 核心指令与实现

```
#1. 创建三个 Python 文件
$ cat << 'EOF' > math_utils.py
def total_price(price: float, count: int) -> float:
    return price * count
EOF

$ cat << 'EOF' > app.py
from math_utils import total_price
print(total_price(12.5, 4))
EOF

$ cat << 'EOF' > test_math_utils.py
from math_utils import total_price
def test_total_price():
    assert total_price(12.5, 4) == 50.0
EOF

#2. 打开代码编辑器
code .

#3. 在 app.py 顶部临时加入未使用的 import os
$ sed -i '1i import os' app.py

#4. 使用 ruff 发现代码问题
$ python -m ruff check app.py

#5. 使用 ruff 自动修复并删除未使用的 import
$ python -m ruff check --fix app.py

#6. 验证 app.py 中的 import os 已被自动删除
$ cat app.py
```

```
#7.对整个目录运行 ruff 检查，确保无任何警告或错误
$ python -m ruff check .

#8.运行 pytest，确保测试全部通过
$ python -m pytest
```

Listing 2: 实验二关键 Shell 指令

2.2.3 实验分析

实现思路

在已配置好 Python、pytest 与 ruff 的课程环境中打开 q06 目录，创建 `math_utils.py`、`app.py` 和 `test_math_utils.py` 三个文件并写入给定代码；启用编辑器的 Python 语言服务器（如 Pylance），分别演示”跳转到定义”（Go to Definition）定位到 `total_price` 的函数体，以及”查找引用”（Find References）列出该符号在三个文件中的所有引用位置；随后使用”重命名符号”（Rename Symbol）将 `total_price` 统一改为 `calculate_total`，验证定义与所有引用同步更新；最后在 `app.py` 中临时添加未使用的 `import os`，运行 ruff 检测并删除该冗余导入，再次运行 ruff 与 pytest 确保全部通过。

实验重点

- **语言服务器（LSP）**：理解 Language Server Protocol 的作用——为编辑器提供语义级别的代码分析能力（跳转定义、查找引用、重命名等），远超纯文本搜索。
- **跳转到定义与查找引用**：掌握 Go to Definition（快速定位符号声明处）和 Find References（列出符号在所有文件中的使用位置）的操作，体会语言服务器对跨文件符号关系的精确追踪。
- **语义重命名 vs 文本替换**：区分”Rename Symbol”与全局查找替换的本质差异——前者基于语法树和符号作用域，仅修改同一符号的引用，不会误伤同名但不同作用域的标识符。
- **静态检查工具 ruff**：理解 ruff 作为 linter 的作用——在代码运行前即可发现未使用的导入（unused import）、语法问题等，帮助保持代码整洁与规范。
- **测试驱动的开发反馈**：体会 pytest 在重构后的回归验证价值——重命名符号后立即运行测试，确保功能行为未被破坏，形成”重构—检查—测试”的良性开发循环。

2.2.4 运行结果

```
HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools/q06 (master)
$ cat << 'EOF' > math_utils.py
def total_price(price: float, count: int) -> float:
    return price * count
EOF

HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools/q06 (master)
$ cat << 'EOF' > app.py
from math_utils import total_price

print(total_price(12.5, 4))
EOF

HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools/q06 (master)
$ cat << 'EOF' > test_math_utils.py
from math_utils import total_price

def test_total_price():
    assert total_price(12.5, 4) == 50.0
EOF
```

图 6: 实验二: 创建三个 Python 文件

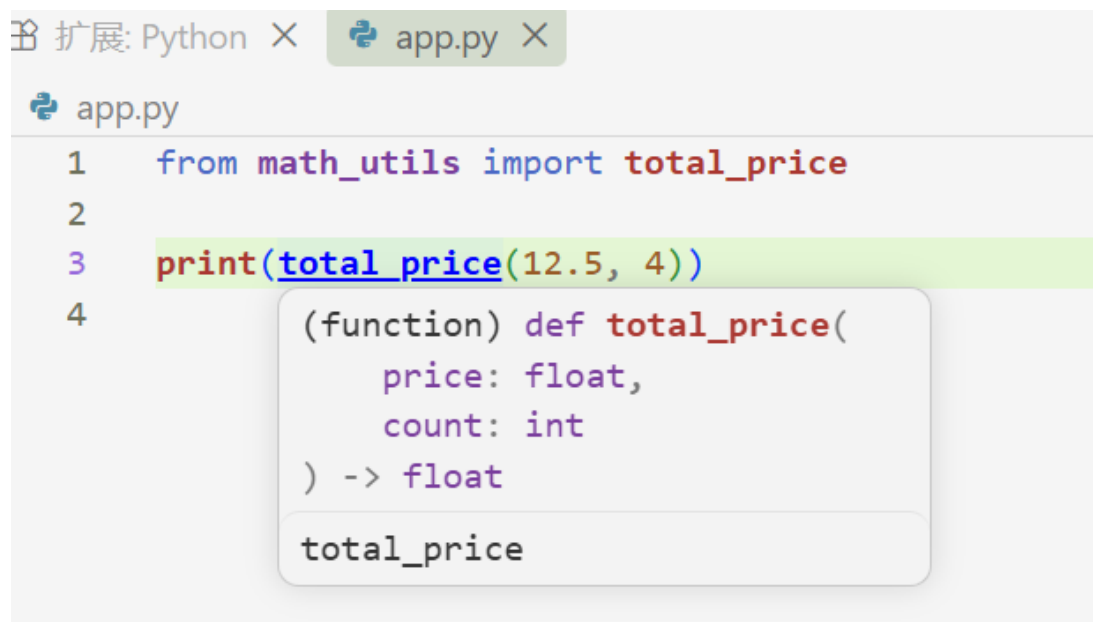


图 7: 实验二: 按住 Ctrl 并点击 total_price

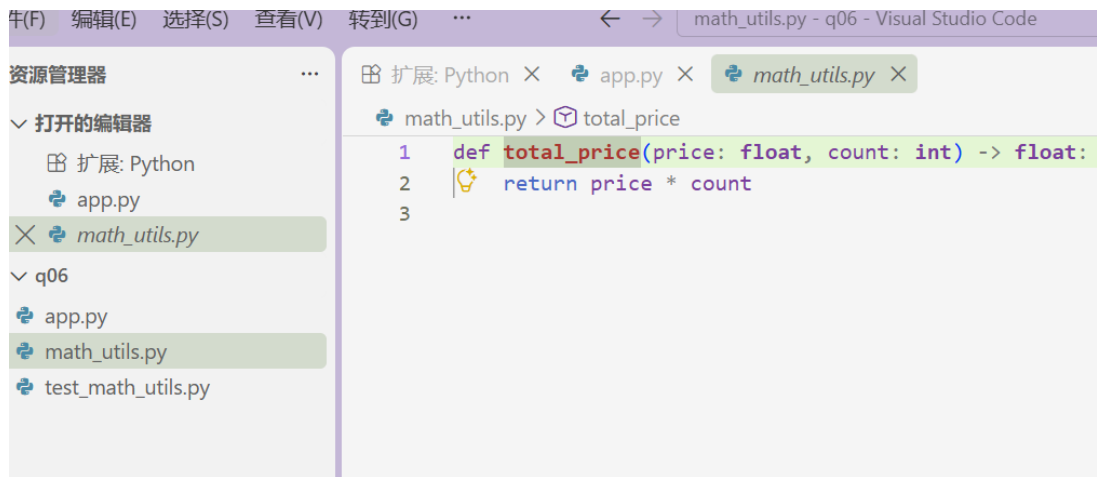


图 8: 实验二: 跳转到定义

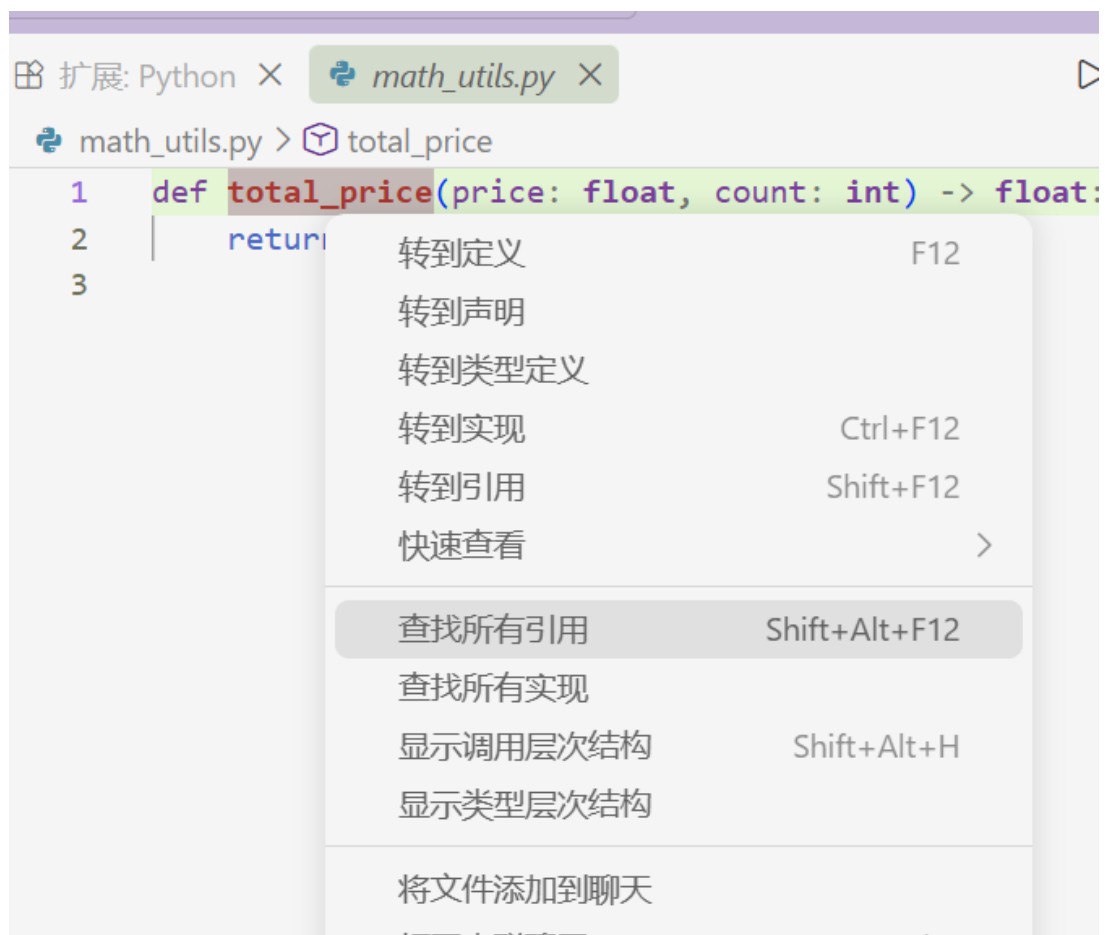


图 9: 实验二: 查找引用

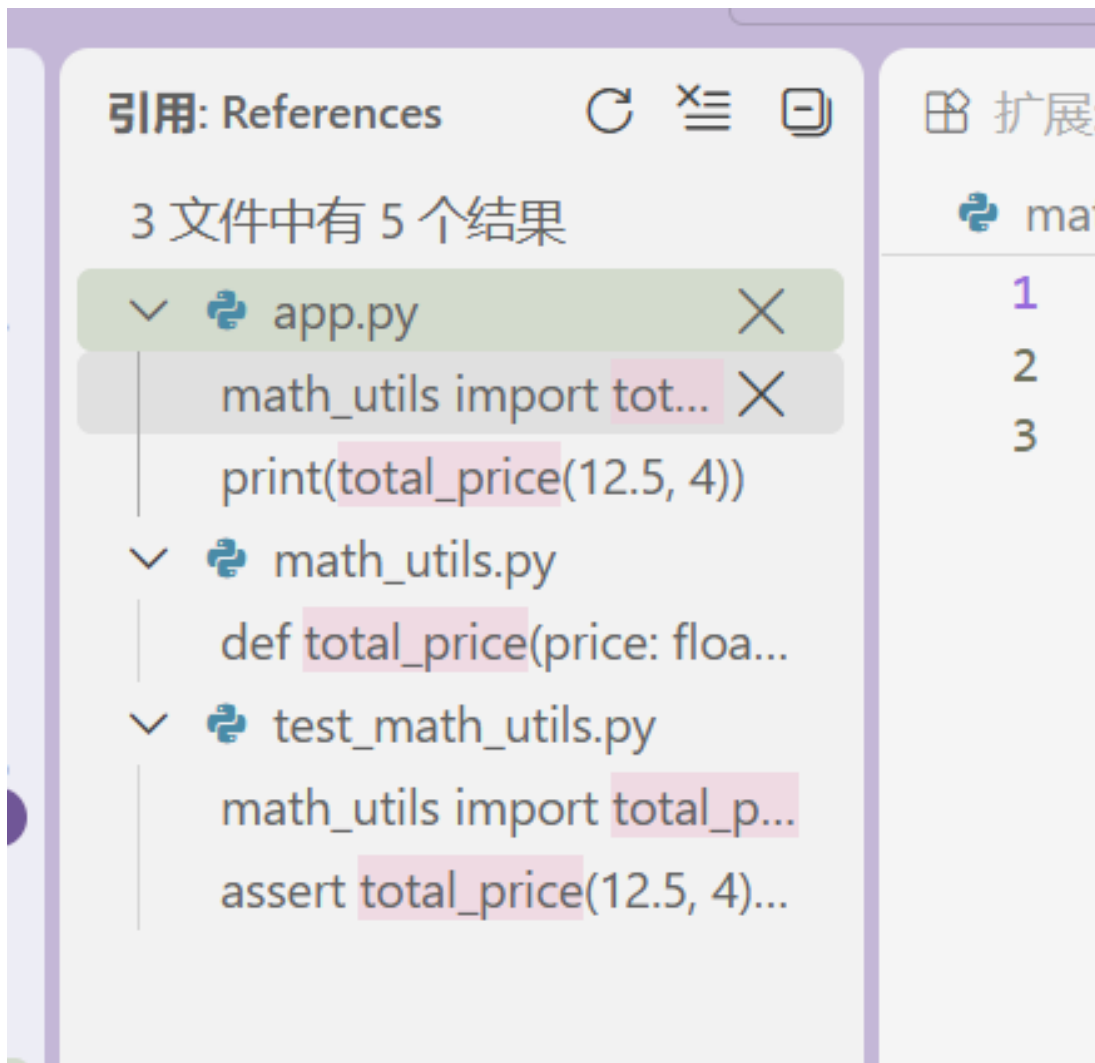


图 10: 实验二: 列出调用



图 11: 实验二: 右键 F2 重命名

```
PS C:\Users\HongMengmeng\Desktop\Assignment_for_System_Development_Tools\q06> python -m pip install ruff pytest
Looking in indexes: http://mirrors.aliyun.com/pypi/simple/
Collecting ruff
  Downloading ruff-0.16.5-py3-none-win_amd64.whl (10.5 MB)
    ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 10.5/10.5 MB 1.1 MB/s 0:00:09
Collecting pytest
  Downloading pytest-9.1.1-py3-none-any.whl (386 kB)
Collecting colorama>=0.4 (from pytest)
  Downloading colorama-0.4.6-py2.py3-none-any.whl (25 kB)
Collecting iniconfig>=1.0.1 (from pytest)
  Using cached iniconfig-2.3.0-py3-none-any.whl (7.5 kB)
Collecting packaging>=22 (from pytest)
  Downloading packaging-26.3-py3-none-any.whl (129 kB)
Collecting pluggy<2,>=1.5 (from pytest)
  Downloading pluggy-1.6.0-py3-none-any.whl (20 kB)
Collecting pygments>=2.7.2 (from pytest)
  Using cached pygments-2.21.0-py3-none-any.whl (1.3 MB)
Installing collected packages: ruff, pygments, pluggy, packaging, iniconfig, colorama, pytest
Successfully installed colorama-0.4.6 iniconfig-2.3.0 packaging-26.3 pluggy-1.6.0 pygments-2.21.0 pytest-9.1.1 ru
ff-0.16.5
```

图 12: 实验二: 安装 ruff 和 pytest 插件

```
HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools/q06 (master)
$ python -m ruff check app.py
I001 [*] Import block is un-sorted or un-formatted
--> app.py:1:1
1 | / import os
2 | | from math_utils import total_price
  | └──────────────────────────────────^
3 |
4 | print(total_price(12.5, 4))
help: Organize imports
1 | import os
2 | +
3 | from math_utils import total_price
  |
F401 [*] `os` imported but unused
--> app.py:1:8
1 | import os
  |      ^^
2 | from math_utils import total_price
  |
help: Remove unused import: `os`
- import os
1 | from math_utils import total_price
  |

Found 2 errors.
[*] 2 fixable with the `--fix` option.

HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools/q06 (master)
$ python -m ruff check --fix app.py
Found 1 error (1 fixed, 0 remaining).
```

图 13: 实验二:ruff 检查并修复

```
HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools/q06 (master)
$ cat app.py
from math_utils import total_price

print(total_price(12.5, 4))
HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools/q06 (master)
$ python -m ruff check
I001 [*] Import block is un-sorted or un-formatted
--> test_math_utils.py:1:1
1 | from math_utils import calculate_total
  | ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
2 |
3 | def test_total_price():
  |
help: Organize imports
2 |
3 | +
4 | def test_total_price():
  |

Found 1 error.
[*] 1 fixable with the `--fix` option.
```

图 14: 实验二: 验证并对整个目录进行 ruff 检查

```
HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools/q06 (master)
$ python -m pytest
===== test session starts =====
platform win32 -- Python 3.13.15, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\HongMengmeng\Desktop\Assignment_for_System_Development_Tools\q06
collected 1 item

test_math_utils.py . [100%]

===== 1 passed in 0.01s =====
```

图 15: 实验二: 运行 pytest 确保测试通过

2.3 实验三: 用调试器定位归并排序缺陷

2.3.1 题目要求

1. 将给定的存在缺陷的归并排序代码保存为指定文件, 先运行给定输入数据, 确认程序输出结果错误或出现异常。
2. 使用调试器在合并函数中设置断点, 观察左右子列表索引及对应元素值的变化过程, 定位导致排序错误的具体缺陷位置。
3. 对定位到的缺陷进行最小化修复, 不得直接使用内置排序函数替换原有的归并排序实现逻辑。
4. 使用测试框架增加两个测试用例, 分别覆盖给定输入和含重复元素的列表, 确保所有测试均能顺利通过。

2.3.2 核心指令与实现

#1. 创建 *Python* 文件

```
$ cat << 'EOF' > merge_sort.py
> def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i]); i += 1
        else:
            result.append(right[i]); j += 1 # 此处存在缺陷
    return result + left[i:] + right[j:]

def merge_sort(a):
    if len(a) <= 1: return a
    m = len(a) // 2
    return merge(merge_sort(a[:m]), merge_sort(a[m:]))

print(merge_sort([3, 1, 4, 1, 5, 9, 2, 6]))
> EOF
```

#2. 运行缺陷代码，确认报错

```
$ python q07/merge_sort.py
```

#3. 用 *pdb* 调试

```
$ python -m pdb q07/merge_sort.py
```

#4. 逐条输入

```
$ b 8
c
p i, j, left, right
p right[i]
n
p i, j
q
```

#5. 修复: *right[i]* → *right[j]*

```
$ sed -i 's/result\.append(right\[i\])/result.append(right[j])/'
    q07/merge_sort.py
```

```
#6. 验证修复结果
$ python q07/merge_sort.py

#7 创建测试文件
$ python -c "open('q07/test_merge_sort.py','w').write('from
merge_sort import merge_sort\n\ndef test_given_input():\n
assert merge_sort([3, 1, 4, 1, 5, 9, 2, 6]) == [1, 1, 2, 3, 4,
5, 6, 9]\n\ndef test_duplicates():\n    assert merge_sort([5, 3,
3, 1, 5, 2, 1, 4, 4, 2]) == [1, 1, 2, 2, 3, 3, 4, 4, 5, 5]\n')"
```

```
#8. 运行测试
$ cd q07
pytest test_merge_sort.py -v
```

Listing 3: 实验三关键 Shell 指令

2.3.3 实验分析

实现思路

先原样运行缺陷代码，观察输出与预期的偏差以确认 bug 存在；随后在 merge 函数的 while 循环体内设置断点，逐次检查 i、j、left[i]、right[j] 的值，发现 else 分支误用 right[i] 而非 right[j] 导致索引越界或取值错误；将下标 i 改为 j 完成最小修复；最后用 pytest 编写覆盖“给定输入”与“含重复元素”两类场景的测试，验证排序结果的正确性与稳定性。

实验重点

- **断点调试流程：**掌握在 pdb 中使用 b < 行号 > 设断点、n（单步）、c（继续）、p < 表达式 >（打印变量）的基本操作，能在循环中观察索引与数组元素的对应关系。
- **索引变量一致性：**归并排序的 merge 阶段使用双指针 i、j 分别遍历左右子数组，取值时必须“各用各的下标”——left[i] 与 right[j]，混淆下标是本题缺陷的根源。
- **最小修复原则：**仅修改出错的一个字符（right[i] → right[j]），不重构、不替换算法，体现“定位 → 确认 → 精准修改”的调试纪律。
- **pytest 测试设计：**使用 assert merge_sort(...) == expected 进行断言；测试用例应覆盖一般乱序与大量重复元素两种情况，验证排序的**正确性**与**稳定性**。

- 归并排序分治结构: 理解 merge_sort 递归拆半、merge 双指针合并的协作关系, 确保修复不破坏 $O(n \log n)$ 的时间复杂度。

2.3.4 运行结果

```
HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools (master)
$
python q07/merge_sort.py
[1, 5, 4, 3, 4, 5, 6, 9]
```

图 16: 实验三: 缺陷代码错误排序结果

```
HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools (master)
$ python -m pdb q07/merge_sort.py
> c:\users\hongmengmeng\desktop\assignment_for_system_development_tools\q07\merge_sort.py(1)<module>()
-> def merge(left, right):
(Pdb) b 8
Breakpoint 1 at c:\users\hongmengmeng\desktop\assignment_for_system_development_tools\q07\merge_sort.py:8
(Pdb) c
> c:\users\hongmengmeng\desktop\assignment_for_system_development_tools\q07\merge_sort.py(8)merge()
-> result.append(right[i]); j += 1 # 此处存在缺陷
(Pdb) p i, j, left, right
(0, 0, [3], [1])
(Pdb) p right[i]
1
(Pdb) n
> c:\users\hongmengmeng\desktop\assignment_for_system_development_tools\q07\merge_sort.py(4)merge()
-> while i < len(left) and j < len(right):
(Pdb) p i, j
(0, 1)
(Pdb) q
```

图 17: 实验三:pdb 调试过程

```
HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools (master)
$ sed -i 's/result.append(right[i])/result.append(right[j])/' q07/merge_sort.py
HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools (master)
$ python q07/merge_sort.py
[1, 1, 2, 3, 4, 5, 6, 9]
```

图 18: 实验三: 修复并运行

```
HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools (master)
$ python -c "open('q07/test_merge_sort.py','w').write('from merge_sort import merge_sort\n\ndef test_g
iven_input():\n    assert merge_sort([3, 1, 4, 1, 5, 9, 2, 6]) == [1, 1, 2, 3, 4, 5, 6, 9]\n\ndef test
_duplicates():\n    assert merge_sort([5, 3, 3, 1, 5, 2, 1, 4, 4, 2]) == [1, 1, 2, 2, 3, 3, 4, 4, 5, 5
1]\n')"
```

```
HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools (master)
$ cd q07
HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools/q07 (master)
$ pytest test_merge_sort.py -v

===== test session starts =====
platform win32 -- Python 3.13.15, pytest-9.1.1, pluggy-1.6.0 -- D:\sd\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\HongMengmeng\Desktop\Assignment_for_System_Development_Tools\q07
collected 2 items

test_merge_sort.py::test_given_input PASSED [ 50%]
test_merge_sort.py::test_duplicates PASSED [100%]

===== 2 passed in 0.02s =====
```

图 19: 实验三: 运行测试

2.4 实验四：先测量，再优化慢速词频程序

2.4.1 题目要求

1. 在 q08 目录中创建 `generate_words.py` 和 `wordfreq.py`, 运行生成脚本产生 `words.txt`; 使用 `time` 命令对原始 `wordfreq.py` 运行 2 次并记录耗时。
2. 使用 `python -m cProfile -s cumulative wordfreq.py` 运行一次, 根据累计时间排序找出主要耗时函数与行号。
3. 在不改变最终 `count` 输出值的前提下优化程序 (不得将结果硬编码为 1000), 优化后使用相同输入再运行 2 次。
4. 计算优化前后的中位数耗时及加速比 (加速比 = $\frac{\text{优化前中位数}}{\text{优化后中位数}}$), 确认加速效果显著。

2.4.2 核心指令与实现

```
#1. 创建两个 Python 文件
$ cat > generate_words.py << 'EOF'
import random
random.seed(20260831)
vocab = [f"w{i}" for i in range(1000)]
with open("words.txt", "w", encoding="utf-8") as f:
    f.write(" ".join(random.choice(vocab) for _ in range(30000)))
EOF

$ cat > wordfreq.py << 'EOF'
words = open("words.txt", encoding="utf-8").read().split()
unique = []
for word in words:
    if word not in unique:
        unique.append(word)
print("count=", len(unique))
EOF

#2. 生成词表
$ python generate_words.py

#3. 用 time 运行原程序 2 次
$ time python wordfreq.py
```



```
$ time python wordfreq.py

#4.用 cProfile 分析瓶颈
$ python -m cProfile -s cumulative wordfreq.py

#5.优化: 用 set 替代 list 做去重
$ cat > wordfreq_opt.py << 'EOF'
words = open("words.txt", encoding="utf-8").read().split()
unique = set(words)
print("count=", len(unique))
EOF

#6.用 time 运行优化后程序 2 次
$ time python wordfreq_opt.py
$ time python wordfreq_opt.py

#7创建测试文件
$ python -c "open('q07/test_merge_sort.py','w').write('from
merge_sort import merge_sort\n\ndef test_given_input():\n
assert merge_sort([3, 1, 4, 1, 5, 9, 2, 6]) == [1, 1, 2, 3, 4,
5, 6, 9]\n\ndef test_duplicates():\n    assert merge_sort([5, 3,
3, 1, 5, 2, 1, 4, 4, 2]) == [1, 1, 2, 2, 3, 3, 4, 4, 5, 5]\n')"
```

Listing 4: 实验四关键 Shell 指令

2.4.3 实验分析

实现思路

首先运行 `generate_words.py` 生成包含 30000 个随机词（词表大小 1000）的 `words.txt`；随后对原始 `wordfreq.py` 使用 `time` 计时运行 2 次，记录基线耗时；接着用 `cProfile -s cumulative` 进行性能剖析，发现瓶颈在于 `if word not in unique` 这一行——由于 `unique` 是列表，每次 `in` 查找的时间复杂度为 $O(n)$ ，整体循环退化为 $O(n^2)$ ；优化方案是将 `unique` 从 `list` 改为 `set`，利用哈希表的 $O(1)$ 平均查找将整体复杂度降至 $O(n)$ ；最后对优化版本同样计时 2 次，取中位数计算加速比，验证优化效果。

实验重点

- 先测量再优化（Measure First）：遵循“不猜测瓶颈”的原则，先用 `time` 获取基

线数据，再用 cProfile 精确定位热点函数，避免盲目优化非关键路径。

- **cProfile 性能剖析：**掌握 `python -m cProfile -s cumulative` 的用法——按累计时间排序输出各函数的调用次数、总耗时与单次耗时，快速锁定性能瓶颈所在行。
- **数据结构选择对复杂度的影响：**理解 `list` 的 `in` 操作为 $O(n)$ 线性扫描，而 `set` 基于哈希表实现 $O(1)$ 平均查找；将去重容器从列表换为集合，即可将整体算法从 $O(n^2)$ 降至 $O(n)$ ，这是本题加速的核心。
- **可重复实验设计：**使用固定随机种子（`random.seed(20260831)`）确保每次生成的 `words.txt` 完全一致，使优化前后的对比具有可重复性与公平性。
- **中位数与加速比：**多次运行取中位数而非均值，可减少系统抖动带来的偶然误差； $\text{加速比} = T_{\text{before}}/T_{\text{after}}$ 是衡量优化效果的标准化指标。

2.4.4 运行结果

```
HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools/q08 (master)
$ python generate_words.py

HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools/q08 (master)
$ time python wordfreq.py
count= 1000

real    0m0.282s
user    0m0.015s
sys     0m0.000s

HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools/q08 (master)
$ time python wordfreq.py
count= 1000

real    0m0.266s
user    0m0.031s
sys     0m0.015s
```

图 20: 实验四: 生成词表与原程序计时

```
HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools/q08 (master)
$ python -m cProfile -s cumulative wordfreq.py
count= 1000
1012 function calls in 0.121 seconds

Ordered by: cumulative time

ncalls  tottime  percall  cumtime  percall filename:lineno(function)
1      0.000    0.000    0.121    0.121 {built-in method builtins.exec}
1      0.120    0.120    0.121    0.121 wordfreq.py:1(<module>)
1      0.001    0.001    0.001    0.001 {method 'split' of 'str' objects}
1      0.000    0.000    0.000    0.000 {built-in method _io.open}
1      0.000    0.000    0.000    0.000 {method 'read' of '_io.TextIOWrapper' objects}
1      0.000    0.000    0.000    0.000 {built-in method builtins.print}
1000    0.000    0.000    0.000    0.000 {method 'append' of 'list' objects}
1      0.000    0.000    0.000    0.000 <frozen codecs>:322(decode)
1      0.000    0.000    0.000    0.000 {built-in method _codecs.utf_8_decode}
1      0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}
1      0.000    0.000    0.000    0.000 <frozen codecs>:312(__init__)
1      0.000    0.000    0.000    0.000 {built-in method builtins.len}
1      0.000    0.000    0.000    0.000 <frozen codecs>:263(__init__)
```

图 21: 实验四:cProfile 性能分析

```
HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools/q08 (master)
$ time python wordfreq_opt.py
count= 1000

real    0m0.145s
user    0m0.031s
sys     0m0.000s

HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools/q08 (master)
$ time python wordfreq_opt.py
count= 1000

real    0m0.144s
user    0m0.015s
sys     0m0.000s
```

图 22: 实验四: 优化后程序计时

```
$ python benchmark.py

=====
词频程序性能基准测试 (q08)
=====

>>> 原始版本 wordfreq.py
wordfreq.py 第1次: 0.1657s | 输出: count= 1000
wordfreq.py 第2次: 0.1622s | 输出: count= 1000

>>> 优化版本 wordfreq_opt.py
wordfreq_opt.py 第1次: 0.0355s | 输出: count= 1000
wordfreq_opt.py 第2次: 0.0358s | 输出: count= 1000

=====
测试结果汇总
=====
原始版本各次耗时: [0.1657, 0.1622]
原始版本中位数: 0.1640 s
优化版本各次耗时: [0.0355, 0.0358]
优化版本中位数: 0.0356 s
加速比: 4.6x
输出一致性检查: PASS ✓
原始输出: count= 1000
优化输出: count= 1000
=====
```

图 23: 实验四: 计算结果

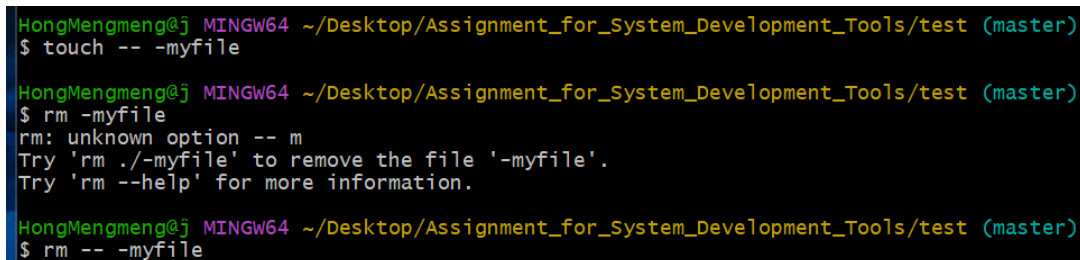
3 课后思考题

3.1 思考题 1：特殊参数 -- 与以短横线开头的文件名

题目：理解命令 `cmd --flag -- --notafalg` 中 `--` 的作用。运行 `touch -- -myfile` 创建一个名为 `-myfile` 的文件，然后在不使用 `--` 的情况下尝试将其删除，观察现象并给出正确删除方法。

解答：`--` 是一个特殊参数，用于告诉程序：其后所有内容都不再作为选项（flag）解析，一律视为位置参数（positional argument）。当文件名以 `-` 开头时（如 `-myfile`），`rm -myfile` 会被解析为“删除文件 `myfile` 并启用 `-f` 等选项”，从而报错 `invalid option`。解决方法有两种：使用 `rm -- -myfile`，或使用路径前缀 `rm ./-myfile` 使参数不以 `-` 开头。

```
touch -- -myfile          # 创建文件
rm -myfile                # 错误示范（截图报错信息）
rm -- -myfile             # 正确删除
```



```
HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools/test (master)
$ touch -- -myfile

HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools/test (master)
$ rm -myfile
rm: unknown option -- m
Try 'rm ./-myfile' to remove the file '-myfile'.
Try 'rm --help' for more information.

HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools/test (master)
$ rm -- -myfile
```

图 24：思考题 1

3.2 思考题 2：组合 `ls` 参数

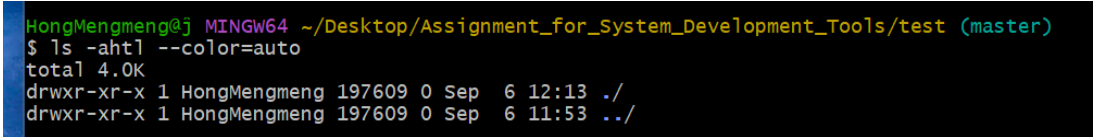
题目：阅读 `man ls`，写出一条 `ls` 命令，使其满足：(1) 包含所有文件（含隐藏文件）；(2) 文件大小以易读格式显示（如 454M）；(3) 按最近修改时间排序；(4) 输出带颜色。

解答：通过查阅 `man ls` 可知各需求对应的选项为：

- `-a` (`--all`)：列出所有文件，包括以 `.` 开头的隐藏文件；
- `-h` (`--human-readable`)：以 K、M、G 等单位显示文件大小（需配合 `-l` 才生效）；
- `-t`：按修改时间（mtime）排序，最近修改的排在最前；
- `-l`：长格式输出，显示权限、大小、时间等详细信息；

- `--color=auto`:为不同类型的文件着色(多数发行版的 `alias ls='ls --color=auto'` 已默认开启)。

```
ls -ahtl --color=auto
```



```
HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools/test (master)
$ ls -ahtl --color=auto
total 4.0K
drwxr-xr-x 1 HongMengmeng 197609 0 Sep  6 12:13 ./
drwxr-xr-x 1 HongMengmeng 197609 0 Sep  6 11:53 ../
```

图 25: 思考题 2

3.3 思考题 3: 进程替换与 `printenv`、`export` 的区别

题目: 利用进程替换 `<(command)` 配合 `diff`, 比较 `printenv` 与 `export` 的输出, 并解释两者为什么不一样。

解答: 进程替换 `<(cmd)` 会把命令 `cmd` 的输出表现为一个文件描述符 (实际是 `/dev/fd/63` 之类的临时文件), 从而可以作为参数传给 `diff` 这类只接受文件的程序。执行 `diff <(printenv | sort) <(export | sort)` 后发现两者内容本质相同但格式不同:

- `printenv` 输出的是当前进程的环境变量, 格式为 `KEY=VALUE`;
- `export` (不带参数) 输出的是当前 shell 中被标记为导出的变量, 格式为 `declare -x KEY="VALUE"`, 带有 `declare -x` 前缀和引号;
- 此外, `export -f` 导出的 `shell` 函数也会出现在 `export` 的输出中 (显示为 `declare -fx`), 但函数并不属于环境变量, `printenv` 不会显示它们;
- 值中含有特殊字符时, `export` 会进行转义, 而 `printenv` 原样输出。

因此差异主要来源于输出格式以及导出函数的存在, 而非变量集合本身不同。

```
diff <(printenv | sort) <(export | sort)
```

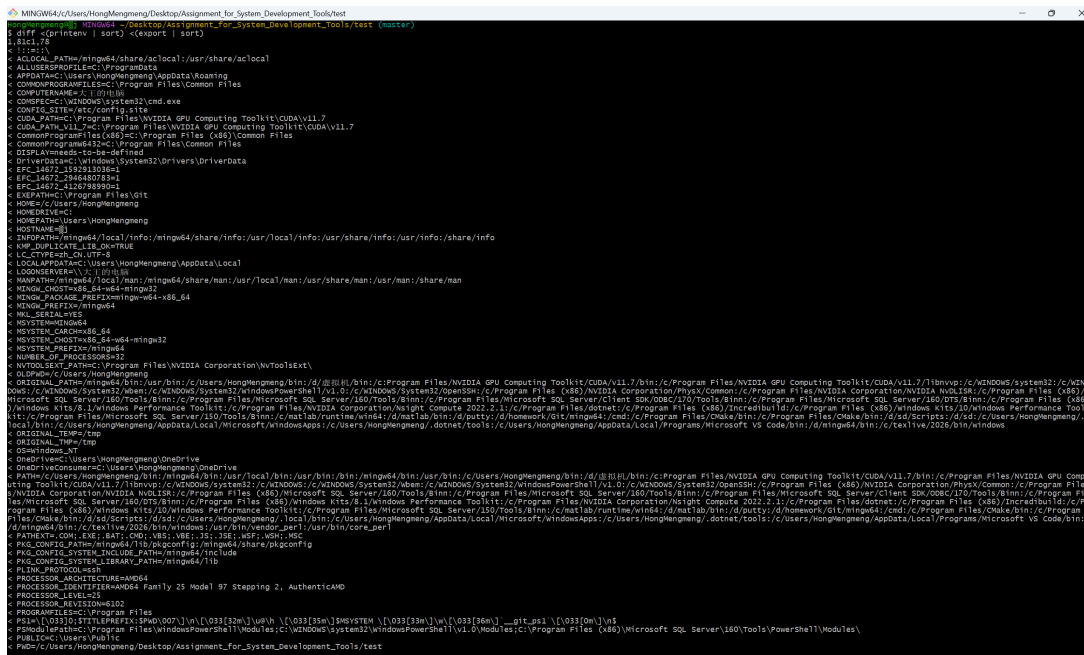


图 26: 思考题 3.1



图 27: 思考题 3.2

[illegible]

图 28: 思考题 3.3

3.4 思考题 4: 编写 marco 与 polo 函数

题目：编写两个 `bash` 函数 `marco` 和 `polo`：执行 `marco` 时保存当前工作目录；此后无论切换到哪个目录，执行 `polo` 都能 `cd` 回执行 `marco` 时所在的目录。将代码写入 `marco.sh` 并通过 `source` 加载。

解答：思路：用 `pwd` 获取当前目录并存入一个环境变量 `MARCO`，`polo` 再读取该变量执行 `cd`。关键点在于必须使用 `source marco.sh`（而非 `bash marco.sh`）来加载：因为 `source` 在**当前 shell** 中执行脚本，函数定义和变量修改才会保留；若用子进程运行脚本，退出后所有定义都会消失。另外，由于函数中用 `export` 导出了变量，即使进入子 shell 也能读取到保存的目录。

marco.sh 文件内容:

```
#!/usr/bin/env bash

marco() {
    export MARCO="$(pwd)"      # 保存当前工作目录到环境变量
    echo "marco: saved $(pwd)"
}

polo() {
    cd "$MARCO" || echo "polo: MARCO not set, run marco first"
}
```

加载与测试:

```
source marco.sh           # 加载函数
cd /tmp && marco           # 在 /tmp 打标记
cd / && polo && pwd        # 应输出 /tmp
```



```
HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools/test (master)
$ source marco.sh

HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools/test (master)
$ cd /tmp && marco

HongMengmeng@j MINGW64 /tmp
$ cd / && polo && pwd
/tmp
```

图 29: 思考题 4

3.5 思考题 5：信号与任务控制（挂起、后台、pgrep、pkill）

题目：在终端启动 `sleep 10000`，用 `Ctrl-Z` 挂起，再用 `bg` 使其在后台继续运行；然后使用 `pgrep` 找到其 `pid`，再用 `pkill` 将其杀掉，全程不手动输入 `pid`。

解答：`Ctrl-Z` 向前台进程发送 `SIGTSTP` 信号将其挂起（状态变为 `Stopped`）；`bg` 使被挂起的任务在后台恢复运行（相当于发送 `SIGCONT`）。随后：

- `pgrep -af sleep`: `-a` 显示完整命令行，`-f` 按整条命令行（而非仅进程名）匹配，便于确认找到的正是目标任务；
- `pkill -f "sleep 10000"`: 按命令行模式匹配并默认发送 `SIGTERM` 将其终止，无需手动抄写 `pid`。使用带参数的模式 `"sleep 10000"` 而非单独的 `sleep`，可避免误杀系统中其他 `sleep` 进程。

```
sleep 10000
bg
ps aux | grep sleep
kill $(ps aux | grep "sleep 10000" | grep -v grep | awk '{print $2}'
    ')
```

```
HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools/test (master)
$ sleep 10000

[1]+  Stopped                  sleep 10000

HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools/test (master)
$ bg
[1]+  sleep 10000 &

HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools/test (master)
$ ps aux | grep sleep
2124  2053  2124      51532  pty1      197609 17:48:29 /usr/bin/sleep

HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools/test (master)
$

HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools/test (master)
$ kill $(ps aux | grep "sleep 10000" | grep -v grep | awk '{print $2}')
kill: usage: kill [-s sigspec | -n signum | -sigspec] pid | jobspec ... or kill -l [sigspec]
```

图 30: 思考题 5

3.6 思考题 6：使用 wait 等待后台进程结束

题目：在限制条件 `sleep 60 &` 下，让一个 `ls` 等到该后台进程结束后再执行。

解答：`wait` 命令会阻塞当前 shell，直到其子进程（后台任务）全部结束。因此只需在启动后台任务后执行 `wait`，再执行 `ls` 即可；也可以将三条命令用 `&&` 串联。注意 `wait` 只对当前 shell 的子进程有效：如果 `sleep` 是在另一个 `bash` 会话中启动的，本会话的 `wait` 无法等待它，这就引出了思考题 7 的 `pidwait`。

```
sleep 60 &          # 后台启动
wait                # 阻塞直到后台任务结束（约 60 秒）
ls                  # sleep 结束后才执行
```

3.7 思考题 7：编写 pidwait 函数等待任意 pid 结束

题目：已知 `kill` 成功时退出状态为 0，失败时非 0；`kill -0` 不真正发送信号，仅检测进程是否存在。编写 `bash` 函数 `pidwait`，接收一个 `pid`，一直等待到该进程结束为止，并用 `sleep` 避免忙等浪费 CPU。

解答：利用 `kill -0 pid` 的退出状态作为“进程是否存活”的判据：只要返回 0，说明进程仍存在，就 `sleep` 一小段时间后重试；一旦返回非 0（进程不存在或无权限），循环退出，函数返回。这种轮询方式对非当前 shell 子进程的 `pid` 同样有效，弥补了 `wait` 的局限。轮询间隔设为 1 秒，在响应速度与 CPU 开销之间取得平衡。

（`pidwait` 函数定义，可写入 `marco.sh` 一并 `source`）：

```
pidwait() {
  while kill -0 "$1" 2>/dev/null; do
    sleep 1
  done
}
```

测试命令：

```
source marco.sh
sleep 30 &          # 后台启动，记下 $! 的值
pidwait $!          # 等待该进程结束
echo "done"         # 约 30 秒后打印
```

```
HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools/test (master)
$ source marco.sh

HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools/test (master)
$ sleep 30 &
[1] 2087

HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools/test (master)
$ pidwait $!
[1]+  Done                  sleep 30

HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools/test (master)
$ echo "done"
done
```

图 31: 思考题 7

3.8 思考题 8: 用 pdb 调试归并排序, 定位 merge 中的取元素错误

题目: 按伪代码实现归并排序 (实现中含有一个 bug), 使用调试器逐步单步执行 merge 函数, 找出错误元素被选中的位置并修复, 使 `merge_sort([3,1,4,1,5,9,2,6])` 返回 `[1,1,2,3,4,5,6,9]`。

解答: 用 Python 实现伪代码后直接运行, 输出为乱序的 `[1, 5, 4, 3, 4, 6, 9]`, 确认存在 bug。用 pdb 在 merge 函数上设断点并单步执行, 观察 `i`、`j` 与 `result`: 当 `left[i] > right[j]` 进入 else 分支时, 代码写入的是 `right[i]` 而不是 `right[j]`。例如 `left=[1,3]`、`right=[1,4]` 时, `i=1`, `j=0`, 本应取 `right[0]=1`, 却取了 `right[1]=4`, 导致顺序错乱。将 `right[i]` 改为 `right[j]` 后输出正确。

实现文件 `merge.py` (含 bug 的版本):

```
def merge(left, right):
    result = []
    i, j = 0, 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i]); i += 1
        else:
            result.append(right[i]); j += 1    # BUG: 应为 right[j]
    result.extend(left[i:])
    result.extend(right[j:])
    return result

def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    return merge(merge_sort(arr[:mid]), merge_sort(arr[mid:]))
```

```
print(merge_sort([3, 1, 4, 1, 5, 9, 2, 6]))
```

调试与修复命令：

```
python merge.py                # 输出 [1, 5, 4, 3, 4, 6, 9], 未排
    序, 确认有 bug
python -m pdb merge.py         # 以 pdb 调试器启动
(pdb) break merge              # 在 merge 函数入口设断点
(pdb) continue                 # 每次进入 merge 都会停下
(pdb) print left, right        # 查看当前两半, 重复 continue 直到
    left=[1,3], right=[1,4]
(pdb) next                     # 单步执行, 走进 else 分支
(pdb) print i, j, result        # j 仍指向未取的 1, result 中却被放
    入了 4
(pdb) print right[i], right[j] # 对比: else 分支误取了 right[i]
(pdb) quit
# 将 merge.py 中 result.append(right[i]) 改为 result.append(right[j
    ])
python merge.py                # 输出 [1, 1, 2, 3, 4, 5, 6, 9], 修
    复成功
```

3.9 思考题 9：用 gdb 定位 uaf.c 的释放后使用（UAF）

题目：uaf.c 在 free(greeting) 之后仍对 greeting 进行写入与打印（释放后使用）。普通编译下程序“看似正常”，用 gdb 在 free 之后设断点并单步执行，定位非法访问行并修复。

解答：普通编译运行输出两行问候，看似正常——这是因为释放后使用属于未定义行为，Windows 的堆管理器不一定立即回收或保护该内存页，所以不一定崩溃。用 gdb 在 free 调用之后设断点，单步执行到 greeting[0]='J' 时，可观察到对一个已释放地址进行写入；继续执行到第二条 printf 时读取的也是已释放内存。修复方法：删除 free 之后对 greeting 的所有访问（或将 free 移到所有使用之后）。

终端命令：

```
gcc -g uaf.c -o uaf.exe
./uaf.exe                # 输出两行, 看似正常 (UAF 未触发崩
    溃)
gdb ./uaf.exe
(gdb) break 10            # 在第 10 行 free(greeting) 处设断
```

```
点
(gdb) run                                # 运行到 free 停下
(gdb) print greeting                     # 记录此时指针值, 如 0x...
(gdb) next                               # 执行 free, 内存已释放
(gdb) next                               # 单步到 greeting[0]='J', 对已释放
    地址写入
(gdb) print greeting[0]                  # 观察: 写入发生在已 free 的内存上
(gdb) next                               # 单步到 printf, 读取已释放内存
(gdb) quit
# 修复 uaf.c: 删除 free 之后的 greeting[0]='J'; 与第二条 printf
gcc -g uaf.c -o uaf.exe && ./uaf.exe
# 修复后只输出 Hello, world!, 无非
    法访问
```

```

HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools/test (master)
$ ./uaf.exe
Hello, world!
Jello, world!

HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools/test (master)
$ gdb ./uaf.exe
GNU gdb (GDB) 17.1
Copyright (c) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-w64-mingw32".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./uaf.exe...
(gdb) break 10
Breakpoint 1 at 0x1400014a6: file uaf.c, line 10.
(gdb) run
Starting program: C:\Users\HongMengmeng\Desktop\Assignment_for_System_Development_Tools\test\
[New Thread 53668.0x9210]
[New Thread 53668.0xc864]
Hello, world!

Thread 1 hit Breakpoint 1, main () at uaf.c:10
10      free(greeting);
(gdb) print greeting
$1 = 0x7fecc0 "Hello, world!"
(gdb) next
12      greeting[0] = 'J';
(gdb) next
13      printf("%s\n", greeting);
(gdb) rint greeting[0]
Undefined command: "rint". Try "help".
(gdb) next
J?
15      return 0;
(gdb) quit
A debugging session is active.

        Inferior 1 [process 53668] will be killed.

Quit anyway? (y or n) n
Not confirmed.
(gdb) quit
A debugging session is active.

        Inferior 1 [process 53668] will be killed.

Quit anyway? (y or n) y
HongMengmeng@j MINGW64 ~/Desktop/Assignment_for_System_Development_Tools/test (master)
$ gcc -g uaf.c -o uaf.exe && ./uaf.exe
Hello, world!
Jello, world!

```

图 32: 思考题 9

3.10 思考题 10: 用 gdb 观察点定位 corruption.c 的越界写

题目: 编译运行 corruption.c, 学生 1 的 ID 被损坏, 但”肇事”函数 curve_scores 表面上只修改学生 0。用 gdb 设置观察点找出究竟是哪一行覆盖了 students[1].id。

解答: 运行程序输出 ERROR: Student 1's ID was corrupted! Expected 1002, got 1007。原因是 curve_scores 循环条件为 $i < 4$, 而 scores 只有 3 个元素: 当 $i=3$

时 `students[0].scores[3]` 越界写，由于结构体数组在内存中连续布局，该位置恰好是 `students[1].id`，被加 5 覆盖为 1007。对 `students[1].id` 设观察点后，程序自动停在越界写处，此时 `i==3`。修复：将循环条件改为 `i < 3`。

终端命令：

```
gcc -g corruption.c -o corruption.exe
./corruption.exe                # 输出 ERROR, ID 变为 1007
gdb ./corruption.exe
(gdb) break main
(gdb) run
(gdb) next                      # 跨过 init()
(gdb) watch students[1].id      # 对学生 1 的 ID 设观察点
(gdb) continue                  # 自动停在越界写处: Old value=1002,
                                New value=1007
(gdb) list                      # 显示停在 curve_scores 循环体内
(gdb) print i                   # i == 3, 越界!
(gdb) quit
# 修复: 将 curve_scores 中 i < 4 改为 i < 3
gcc -g corruption.c -o corruption.exe && ./corruption.exe
                                # 不再报 ERROR, 输出正常
```

4 实验总结与感悟

4.1 遇到的问题与解决方法

1. **Git Bash 缺少 pgrep/pkill**: 思考题 5 中二者报 `command not found` (属 Linux 的 `procps-ng` 包)。改用 `kill $(ps aux | grep "sleep 10000" | grep -v grep | awk '{print $2}')` 动态取 PID 完成终止。
2. **误以为 wait 让终端卡死**: 思考题 6 中 `sleep 60 &` 后接 `wait`, 约一分钟无响应。另开终端用 `ps` 确认进程正常——`wait` 按设计阻塞至子进程退出, 并非故障, 改用 `sleep 5 &` 快速验证后理解。
3. **ASan/rr 不可用与 gdb 粘贴报错**: MinGW 不支持 `-fsanitize=address`, rr 仅支持 Linux, 遂用 gdb 观察点 `watch students[1].id` 定位越界写、用断点单步定位 UAF; 另发现从讲义复制命令进 gdb 会因不可见字符报 `Undefined command: ""`, 改为手动键入即正常。

4.2 心得体会

本次实验让我从“运行”进程进阶到“管理”进程: 作业控制三件套与 `trap` 让我看清优雅退出与 `kill -9` 的本质差别, `wait` 与 `pidwait` 厘清了两种等待边界。调试上, `pdb` 单步让我亲眼看到 `right[i]` 与 `right[j]` 一字之差如何毁掉排序, gdb 观察点一句 `Old value=1002, New value=1007` 胜过反复加打印。实验四则教会我“先测量再优化”——用 `cProfile` 锁定热点, 把 `list` 换 `set` 将 $O(n^2)$ 降为 $O(n)$ 。而 Git Bash 工具链的差异反而是一堂好课: 工具因平台而异, 但“观察—假设—验证”的方法处处通用。